Bölüm 10 Nesnelerde Düşünme

Liang, Java Programlama'ya Giriş, Onuncu Baskı, (c) 2015 Pearson Education, Inc. Tüm hakları saklıdır.

Motivasyonlar

Nesne yönelimli programlamanın avantajlarını görüyorsunuz önceki bölümden. Bu bölüm gösterecektir Nesne yönelimli paradigma kullanarak sorunları nasıl çözeceğiniz.

Hedefler

Sınıf soyutlamasını yazılım geliştirmek için uygulamak (§10.2).

Prosedürel paradigma ile nesne yönelimli paradigma arasındaki farkları keşfetmek için yönlendirilmiş paradigma (§10.3).

- Sınıflar arasındaki ilişkileri keşfetmek (§10.4).

Objektif yönelimli paradigma kullanarak programlar tasarlamak (§§10.5–10.6).

Temel değerler için sarmalayıcı sınıfları kullanarak nesneler oluşturmak (**Byte**, **Kısa**, **Tam sayı**, **Uzun**, **Float**, **Double**, **Karakter, ve Boolean**) (§10.7).

Temel türler ile sarmalayıcı sınıf türleri arasında otomatik dönüşüm kullanarak programlamayı basitleştirmek (§10.8).

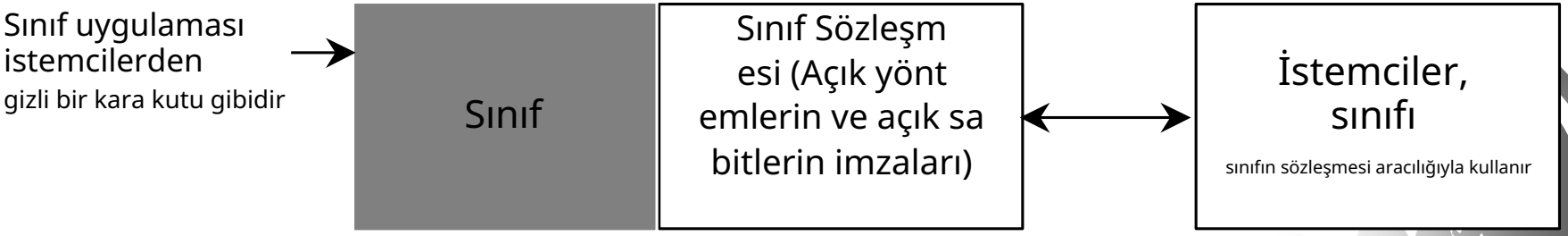
Çok büyük sayıları hesaplamak için **BigInteger** ve **BigDecimal** sınıflarını kullanmak rastgele hassasiyetlere sahip sayılar (§10.9).

String sınıfını değişmez dizeleri işlemek için kullanmak (§10.10).

StringBuilder ve**StringBuffer** sınıflarını kullanarak değiştirilebilir strings (§10.11).

Sınıf Soyutlama ve Kapsülleme

Sınıf soyutlaması, sınıf uygulamasını ayırmak anlamına gelir.
class kullanımından. Sınıfın yaratıcısı sağlar
Bir sınıfın tanımı ve kullanıcının sınıfın nasıl kullanı
labileceği hakkında bilgi verin. Sınıfın kullanıcısının
Sınıfın nasıl uygulandığını bilmek.
uygulama kapsüllenmiş ve kullanıcıdan gizlenmiştir.



Kredi Sınıfını Tasarlamak

Kredi	
-yıllıkFaizOranı: double	Kredinin yıllık faiz oranı (varsayılan: 2.5).
-yılSayısı: int	Kredi için yıl sayısı (varsayılan: 1)
-krediMiktarı: double	Kredi miktarı (varsayılan: 1000).
-krediTarihi: Tarih	Bu kredinin oluşturulma tarihi.
+Kredi()	Varsayılan bir Kredi nesnesi oluşturur.
+Kredi(yıllıkFaizOranı: double, yılSayısı: int, krediMiktarı: double)	Belirtilen faiz oranı, yıl ve kredi miktarı ile bir kredi oluşturur.
+yıllıkFaizOranıAl(): double	Bu kredinin yıllık faiz oranını döndürür.
+yılSayısıAl(): int	Bu kredinin yıl sayısını döndürür.
+krediMiktarıAl(): double	Bu kredinin miktarını döndürür.
+krediTarihiAl(): Tarih	Bu kredinin oluşturulma tarihini döndürür.
+yıllıkFaizOranıBelirle(yıllıkFaizOranı: double): void	Bu krediye yeni bir yıllık faiz oranı ayarlar.
+yılSayısıBelirle(yılSayısı: int): void	Bu krediye yeni bir yıl sayısı ayarlar.
+krediMiktarıBelirle(krediMiktarı: double): void	Bu krediye yeni bir miktar ayarlar.
+aylıkÖdemeAl(): double	Bu kredinin aylık ödemesini döndürür.
+toplamÖdemeAl(): double	Bu kredinin toplam ödemesini döndürür.

Kredi

TestKrediSınıfı

Çalıştır

Nesne Yönelimli Düşünme

Bölüm 1-8 temel programlamayı tanıttı
döngüler, yöntemler ve kullanarak problem çözme teknikleri

Bu tekniklerin çalışmaları, nesne yönelimli programlama için sağlam bir
basamak oluşturur. Sınıflar
inşa etmek için daha fazla esneklik ve modülerlik sağlamak
yeniden kullanılabilir yazılım. Bu bölüm, bir çözümü geliştirmektedir.
nesne yönelimli
yaklaşım. İyileştirmelerden, prosedürel

programlama ve nesne yönelimli programlama ve görün
nesneleri ve sınıfları kullanarak yeniden kullanılabilir kod geliştirme avantajlarını
görün.

BMI Sınıfı

Bu veri alanları için get yöntemleri sınıfta sağlanmıştır, ancak UML diyagramında kısalık açısından atlanmıştır.

BMI
-isim: String -yaş: int -ağırlık: double -yükseklik: double
+BMI(ad: String, yaş: int, kilo: double, yükseklik: double) +BMI(ad: String, kilo: double, yükseklik: double) +getBMI(): double +getStatus(): String

Kişinin adı.

Kişinin yaşı.

Kişinin ağırlığı pound cinsinden.

Kişinin boyu inç cinsinden.

Belirtilen ad, yaş, kilo ve yükseklik ile bir BMI nesnesi oluşturur.

Belirtilen ad, kilo, yükseklik ve varsayılan bir yaş ile bir BMI nesnesi oluşturur.

20.

BMI'yi döndürür

BMI durumunu döndürür (örneğin, normal, aşırı kilolu, vb.)

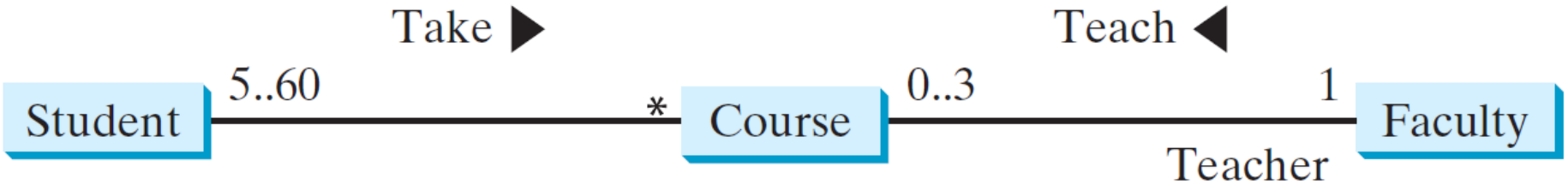
BMI

UseBMIClass

Çalıştır

Nesne Bileşimi

Bileşim aslında bir toplama ilişkisinin özel bir durumudur toplama modelleri *sahiplik* ilişkilerini temsil eder ve iki nesne arasında bir sahiplik ilişkisini gösterir. Sahip nesne *toplayan nesne* olarak adlandırılır ve onun sınıfı *toplayan sınıf* olarak adlandırılır. Konu nesnesi *toplanan nesne olarak adlandırılır ve onun sınıfı toplanan sınıf olarak adlandırılır.*



Sınıf Temsili

Bir toplama ilişkisi genellikle toplama sınıfındaki bi r veri alanı olarak temsil edilir. Örneğin, ilişki Şekil 10.6 aşağıdaki gibi temsil edilebilir:

```
public class Name {
    ...
}
```

Toplanmış sınıf

```
public class Öğrenci {
    private İsim isim;
    private Adres adres;

    ...
}
```

Toplama sınıfı

```
public class Address {
    ...
}
```

Toplanmış sınıf

Toplama veya Bileşim

Toplama ve bileşim ilişkileri,

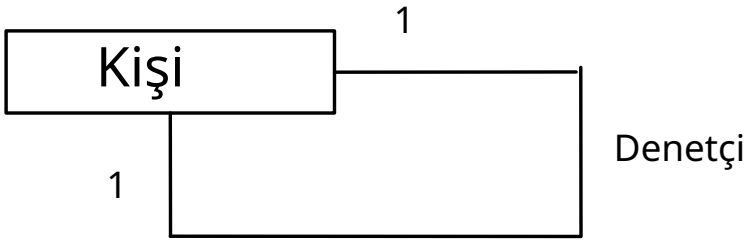
benzer şekillerde sınıflar kullanılarak temsil edildiğinden, birçok metin

bunları ayırt etmez ve her ikisine de bileşim der.

Aynı Sınıf Arasında Toplama

Toplama, aynı sınıfın nesneleri arasında var olabilir.

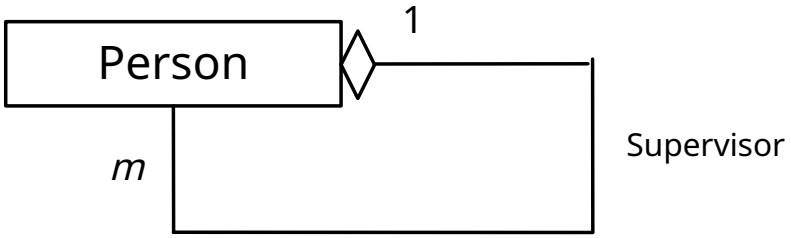
Örneğin, bir kişinin bir süpervizörü olabilir.




```
public class Person {
  // Verinin türü sınıfın kendisidir
  private Person supervisor;
  ...
}
```

Aynı Sınıf Arasında Toplama

Bir kişinin birden fazla süpervizörü varsa ne olur?




```
public class Person {
   …
  private Person[]  supervisors;
}
```

Örnek: Kurs Sınıfı

Kurs	
-kursAdı: String	Course'un adı.
-öğrenciler: String[]	Course için öğrencileri saklamak için bir dizi.
-öğrenciSayısı: int	Öğrenci sayısı (varsayılan: 0).
+Course(courseName: String)	Belirtilen adıyla bir kurs oluşturur.
+getCourseName(): String	Kurs adını döndürür.
+addStudent(student: String): void	Kursa yeni bir öğrenci ekler.
+dropStudent(student: String): void	Kursdan bir öğrenciyi çıkarır.
+getStudents(): String[]	Kursdaki öğrencileri döndürür.
+getNumberOfStudents(): int	Kursdaki öğrenci sayısını döndürür.

Çalıştır

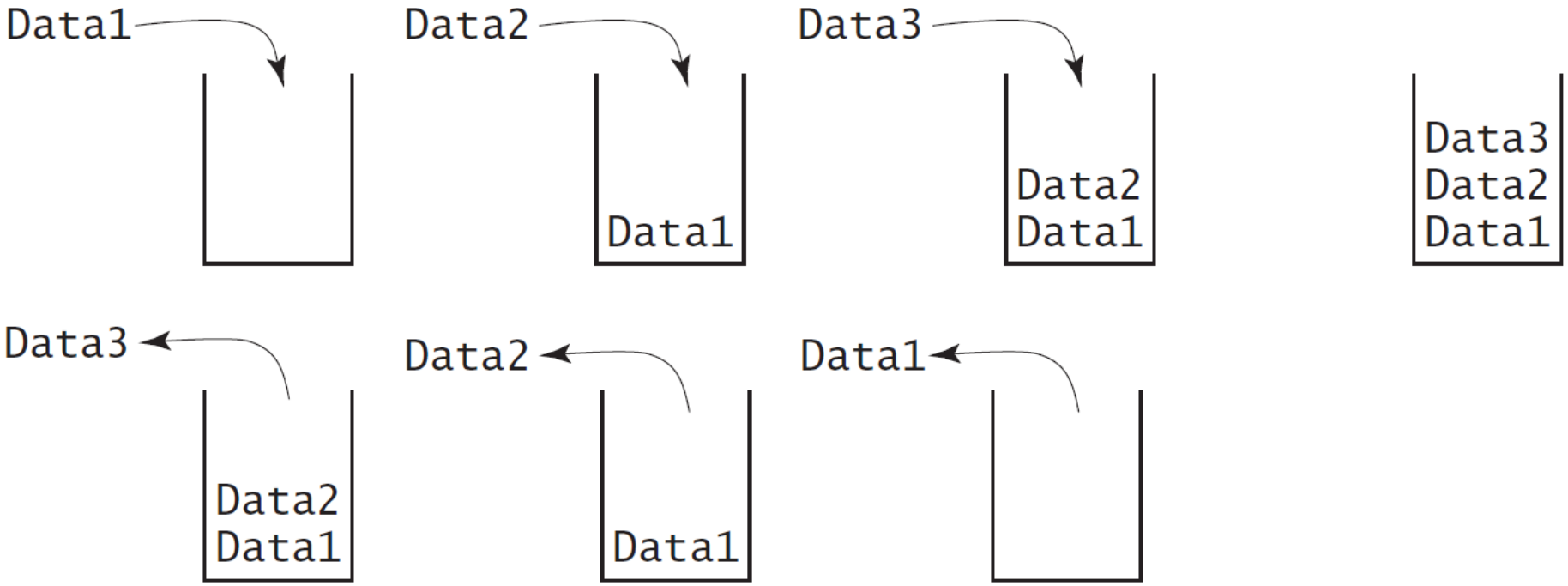
Örnek: The `StackOfIntegers` Sınıfı

StackOfIntegers	
-elemanlar: int[]	Yığın içinde tam sayıları depolamak için bir dizi.
-boyut: int	Yığın içindeki tam sayıların sayısı.
+StackOfIntegers()	Varsayılan kapasitesi 16 olan boş bir yığın oluşturur.
+StackOfIntegers(kapasite: int)	Belirtilen kapasiteye sahip boş bir yığın oluşturur.
+boş(): boolean	Yığın boşsa true döner.
+gözat(): int	Yığının en üstündeki tam sayıyı yığından çıkarmadan döner.
+it(value: int): int	Yığının en üstüne bir tam sayı depolar.
+çıkar(): int	Yığının en üstündeki tam sayıyı kaldırır ve döner.
+getSize(): int	Yığındaki eleman sayısını döner.

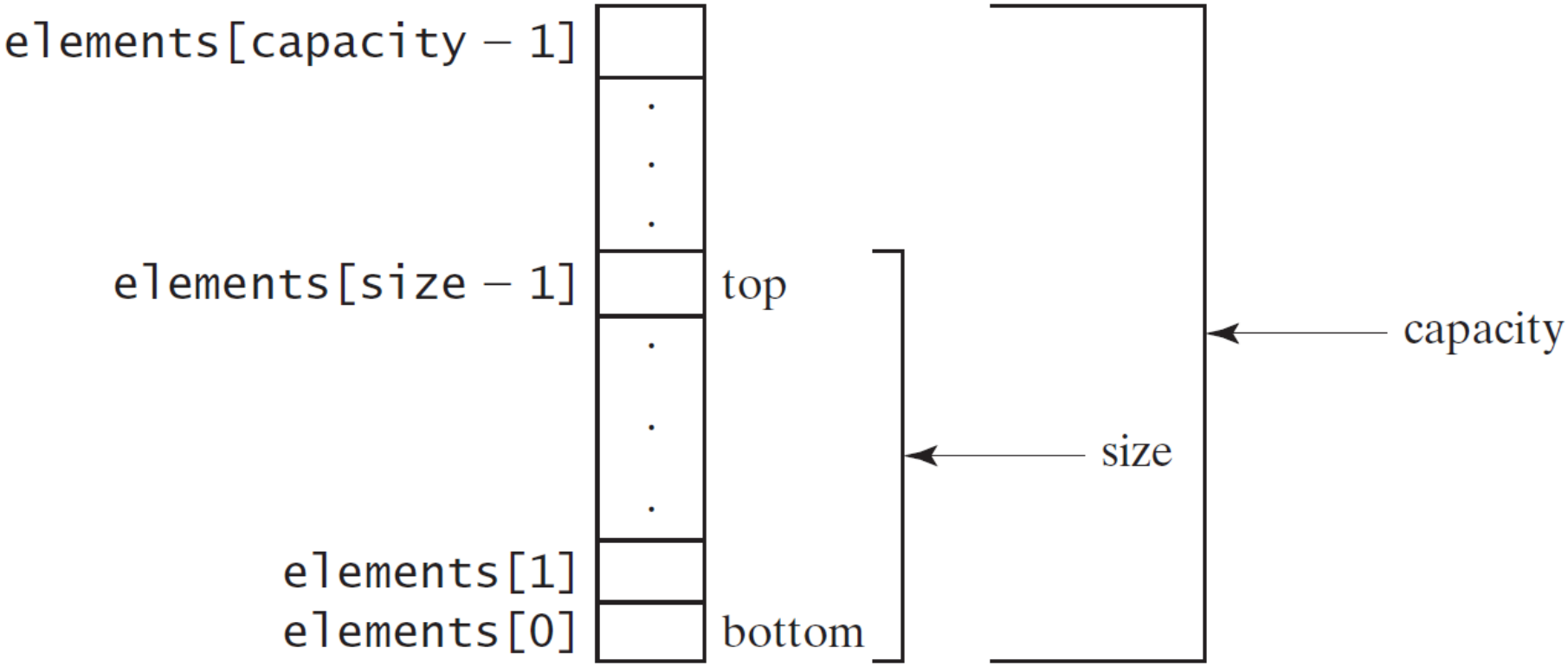
TestStackOfIntegers

Çalıştır

StackOfIntegers Sınıfını Tasarlamak



Uygulama
`StackOfIntegers` Sınıfı



StackOfIntegers

Wrapper Sınıfları

- Boolean
- Karakter
- Kısa
- Bayt
- Tam Sayı
- Uzun
- Float
- Double

NOT: (1) Sarmalayıcı sınıfların parametresiz yapıcıları yoktur. (2) Tüm sarmalayıcıların örnekleri sınıflar değişmezdir, yani onların iç değerler değiştirilemez nesneler oluşturulduktan sonra.

Integer ve Double Sınıfları

java.lang.Integer
-değer: int +MAX_VALUE: int +MIN_VALUE: int
+Integer(value: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Integer): int +toString(): String +valueOf(s: String): Integer +valueOf(s: String, radix: int): Integer +parseInt(s: String): int +parseInt(s: String, radix: int): int

java.lang.Double
-value: double +MAX_VALUE: double +MIN_VALUE: double
+Double(değer: double) +Double(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longValue(): long +floatValue(): float +doubleValue(): double +compareTo(o: Double): int +toString(): String +valueOf(s: String): Double +valueOf(s: String, radix: int): Double +parseDouble(s: String): double +parseDouble(s: String, radix: int): double

Integer Sınıfı ve Double Sınıfı

- Yapıcılar
  - Sınıf Sabitleri `MAX_VALUE`, `MIN_VALUE`
- Dönüşüm Yöntemleri

Sayısal Sarmalayıcı Sınıfı Yapıcıları

Bir sarmalayıcı nesneyi ya bir

ilkel veri türü değerinden ya da bir string'den oluşturabilirsiniz.

sayısal değeri temsil etme. Integer ve Do uble için yapıcılar:

```
public Integer(int value)

public Integer(String s)

public Double(double value)

public Double(String s)
```

Sayısal Sarmalayıcı Sınıf Sabitleri

Her sayısal sarmalayıcı sınıfın sabitleri vardır MAX_VALUE ve MIN_VALUE. MAX_VALUE karşılık gelen

ilkel veri türünün maksimum değerini temsil eder. Byte, Short, Integer ve Long için,

MIN_VALUE, minimum byte, short, int değerini temsil eder, ve uzun değerler. Float ve Double için, MIN_VALUE minimum *pozitif* float ve double değerlerini temsil eder.

Aşağıdaki ifadeler maksimum tam sayıyı (2.147.483.647), minimum pozitif float (1.4E-45), ve maksimum double kayan nokta sayısını

(1.79769313486231570e+308d) gösterir.

Dönüşüm Yöntemleri

Her sayısal sarmalayıcı sınıf, Number sınıfında tanımlanan abstract yöntemler olan doubleValue, floatValue, intValue, longValue ve shortValue'ı uygular. Bu yöntemler nesneleri ilkel tip değerleri.

Statik valueOf Yöntemleri

Sayısal sarmalayıcı sınıflar, yararlı bir sınıf yöntemi olan valueOf(String s) yöntemine sahiptir. Bu yöntem, belirtilen dize tarafından temsil edilen değere göre başlatılmış yeni bir nesne oluşturur. Örneğin:

```
Double doubleObject = Double.valueOf("12.4");
Integer integerObject = Integer.valueOf("12");
```

Dizeleri Sayılara Dönüştürme Yön temleri

Bir sayısal dizeyi bir

int değeri ve parseDouble metodu

Bir sayısal dizeyi bir double değere ayrıştırmak iç in çift sınıf. Her sayısal sarmalayıcı sınıfın, bir s ayısal dizeyi uygun bir sayısal değere ayrıştırmak için iki aşırı yüklenmiş ayrıştırma yöntemi vardır. değer.

Temel Arasında Otomatik Dönüşüm Türler ve Sarmalayıcı Sınıf Türleri

JDK 1.5, ilkel türlerin ve sarmalayıcı sınıfların otomatik olarak dönüştürülmesine olanak tanır. Örneğin, (a) bölümündeki aşağıdaki ifade (b) bölümündeki gibi basitleştirilebilir:

```
Integer[] intArray = {new Integer(2),
    new Integer(4), new Integer(3)};
```

(a)

Eşdeğer

```
Integer[] intArray = {2, 4, 3};
```

Yeni JDK 1.5 kutulama

(b)

```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Kutudan çıkarma

BigInteger ve BigDecimal

Eğer çok büyük tam sayılarla veya

yüksek hassasiyetli kayan nokta değerleriyle hesaplama yapmanız gerekiyorsa,

java.math paketindeki BigInteger ve BigDecimal sınıfları. İkisi de *değişmez*. Her ikisi de Number sınıfını genişletin ve

Comparable arayüzünü uygulayın.

BigInteger ve BigDecimal

```
BigInteger a = new BigInteger("9223372036854775807");
BigInteger b = new BigInteger("2");
BigInteger c = a.multiply(b); // 9223372036854775807 * 2
System.out.println(c);
```

LargeFactorial

Çalıştır

```
BigDecimal a = new BigDecimal(1.0);
BigDecimal b = new BigDecimal(3);
BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);
System.out.println(c);
```

String Sınıfı

- Bir Dize Oluşturma:

  String message = "Java'ya Hoş Geldiniz";

  String message = new String("Java'ya Hoş Geldiniz");

  `String s = new String();`

- Dize uzunluğunu alma ve

  bir dizideki bireysel karakterleri alma

- Dize Birleştirme (concat)
- Alt Dizeler (substring(index), substring(start, end))
- Karşılaştırmalar (equals, compareTo)
- Dize Dönüşümleri

Bir Dizede Bir Karakter veya Alt Dize Bulma

- Dize ve Dizi Arasındaki Dönüşümler
- Karakterleri ve Sayısal Değerleri Dizeye Dönüştürme

Dizeleri Oluşturma

```
String newString = new String(stringLiteral);
```

```
String message = new String("Java'ya Hoş Geldiniz");
```

Dizeler sıkça kullanıldığından, Java bir kısa yol başlatıcısı sağlar:

```
String message = "Java'ya Hoş Geldiniz";
```

Dizeler Değiştirilemezdir

A String nesnesi değiştirilemez; içeriği değiştirilemez. Aşağıdaki kod, string'in içeriğini değiştirir mi?

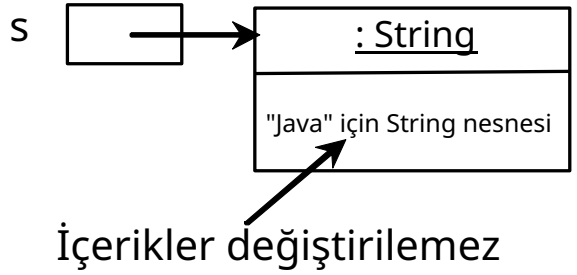
```
String s = "Java";
s = "HTML";
```

animasyon

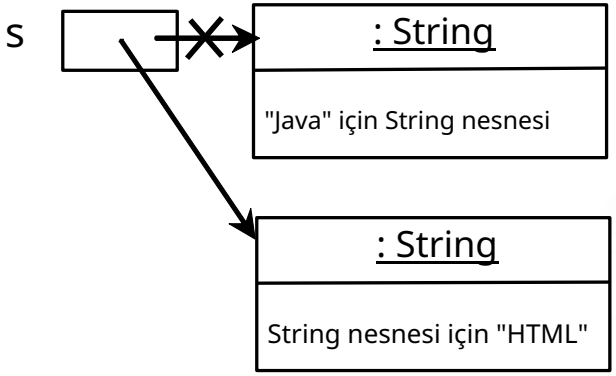
Kodu İzle

```
String s = "Java";
s = "HTML";
```

Çalıştırdıktan sonra `String s = "Java";`



After executing `s = "HTML";`

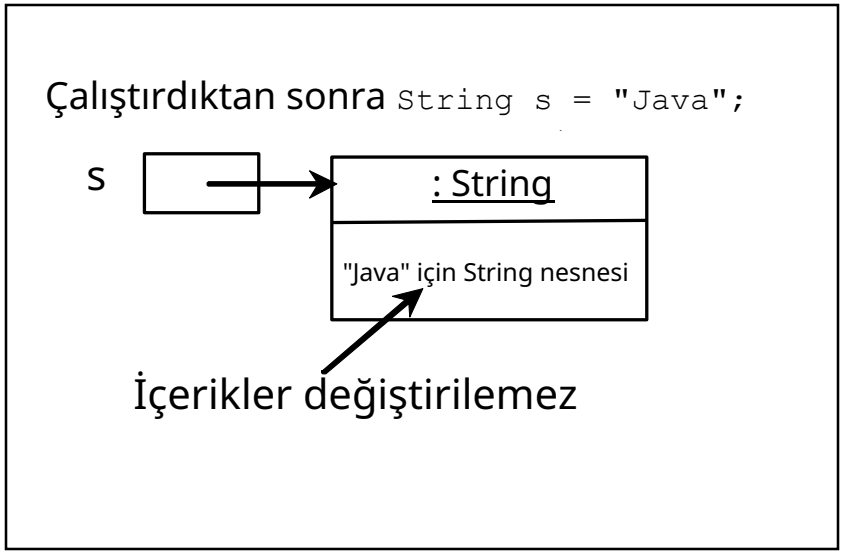


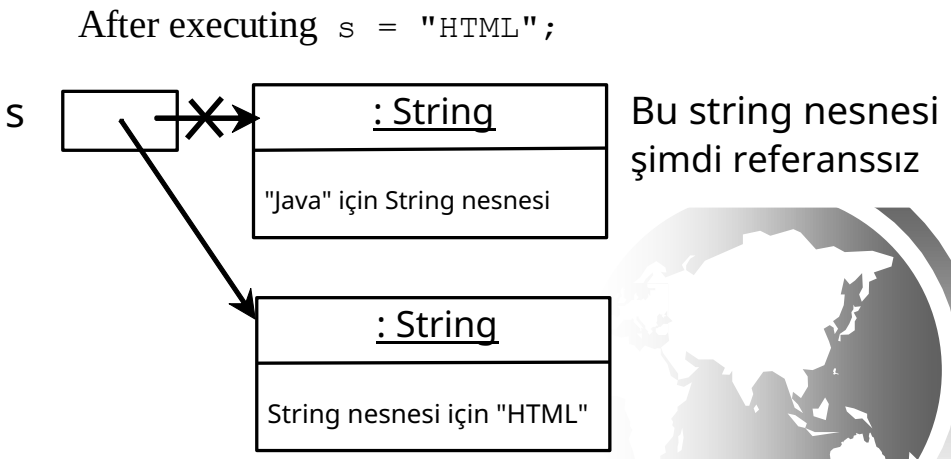
animasyon

Kodu İzle

String s = "Java";

s = "HTML";





Interned Strings

Dizeler değiştirilemez olduğundan ve sıkça kullanıldığından, verimliliği artırmak ve bellek tasarrufu sağlamak için, JVM aynı karakter dizisine sahip string literalleri için benzersiz bir örnek kullanır. Böyle bir örnek called*interned*. Örneğin, aşağıdaki ifadeler:

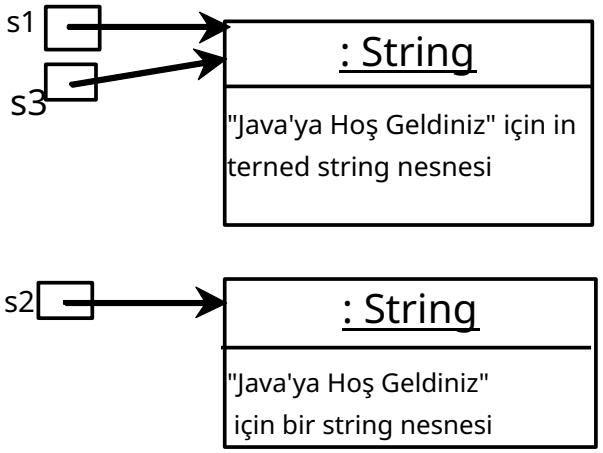
Örnekler

```
String s1  ="Java'ya Hoş Geldiniz";

String s2  =new String("Java'ya Hoş Geldiniz");

String s3 =  "Java'ya Hoş Geldiniz";

System.out.println("s1 == s2 durumu " + (s1 == s2));
System.out.println("s1 == s3 durumu " + (s1 == s3));
```



görüntü

s1 == s yanlıştır

s1 == s3 doğrudur

Yeni bir nesne oluşturulur eğer yeni operatör.

String başlatıcısını kullanıyorsanız, yeni bir nesne, eğer internetteki nesne zaten oluşturulmuş.

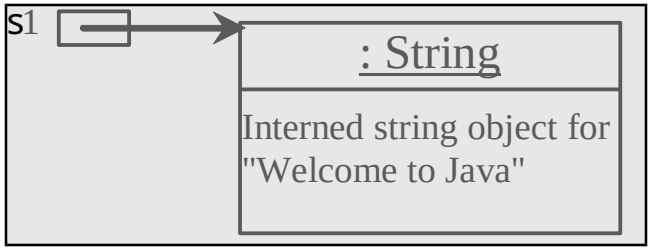
animasyon

Kodu İzle

```
String s1 = "Welcome to Java";
```

String s2 = new String("Java'ya Hoş Geldiniz");

String s3 = "Java'ya Hoş Geldiniz";

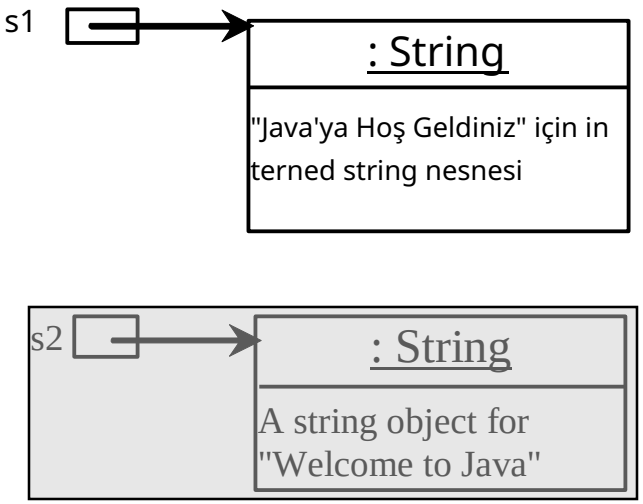


Kodu İzle

String s1 = "Java'ya Hoş Geldiniz";

```
String s2 = new String("Welcome to Java");
```

String s3 = "Java'ya Hoş Geldiniz";

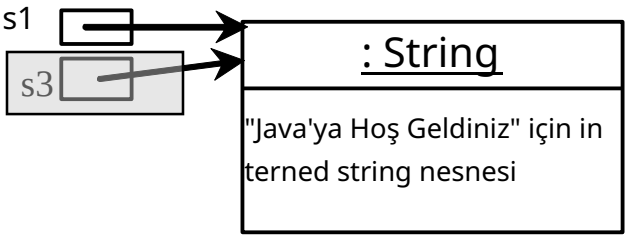


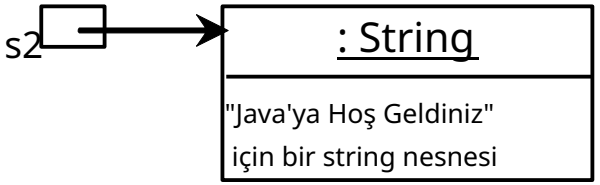
Kodu İzle

String s1 = "Java'ya Hoş Geldiniz";

String s2 = new String("Java'ya Hoş Geldiniz");

```
String s3 = "Welcome to Java";
```





Dizeleri Değiştirme ve Bölme

java.lang.String
+replace(eskiKarakter: char, yeniKarakter: char): String
+replaceFirst(eskiString: String, yeniString: String): String
+replaceAll(eskiString: String, yeniString: String): String
+split(delimiter: String): String[]

Bu string içindeki tüm eşleşen karakterleri yeni karakterle değiştiren yeni bir string döndürür.

Bu string içindeki ilk eşleşen alt dizeyi yeni alt dize ile değiştiren yeni bir string döndürür.

Bu string içindeki tüm eşleşen alt dizeleri yeni alt dize ile değiştiren yeni bir string döndürür.

Ayırıcı ile bölünmüş alt dizelerden oluşan bir string dizisi döndürür.

Örnekler

"Welcome".replace('e', 'A') yeni bir string döndürür, WAlcomA.

"Welcome".replaceFirst("e", "AB") yeni bir string döndürür, WABlcome.

"Welcome".replace("e", "AB") yeni bir string döndürür, WABlcomAB.

Welcome".replace("el", "AB") yeni bir dize döndürür, WABcome.

Bir Dizeyi Bölme

```
String[] tokens = "Java#HTML#Perl".split("#", 0);
for (int i = 0; i < tokens.length; i++)
  System.out.print(tokens[i] + " ");
```

gösterir

Java HTML Perl

Desenlerle Eşleştirme, Değiştirme ve Bölme

Bir dizeyi bir desen belirterek eşleştirebilir, değiştirebilir veya bölebilirsiniz. Bu son derece yararlı ve güçlü bir özelliktir, yaygın olarak bilinen *düzenli ifade*. Düzenli ifade, başlangıç öğrencileri için karmaşıktır. Bu nedenle, iki basit desen vardır bu bölümde kullanılmıştır. Lütfen Ek III.F, “Düzenli İfadeler,” daha ileri çalışmalar için.

```
"Java".matches("Java");
"Java".equals("Java");

"Java eğlencelidir".matches("Java.*");
"Java havalıdır".matches("Java.*");
```

Desenlerle Eşleştirme, Değiştirme ve Bölme

replaceAll, replaceFirst ve split yöntemleri bir düzenli ifade ile kullanılabilir. Örneğin, aşağıdaki ifade "a+b$#c" içindeki $, + veya # karakterlerini yeni bir dize ile değiştirir. string NNN.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");
System.out.println(s);
```

Burada düzenli ifade [$+#] bir deseni belirtir ki matches $, +, or #. So, the output is aNNNbNNNNNNc.

Desenlerle Eşleştirme, Değiştirme ve Bölme

Verilen ifade, dizeyi bir dizi dizeye böler bazı noktalama işaretleriyle sınırlıdır.

```
String[] tokens = "Java,C?C#,C++".split("[.,:;?]");

for (int i = 0; i < tokens.length; i++)
  System.out.println(tokens[i]);
```

Karakterleri ve Sayıları

Dizelere Dönüştür

String sınıfı, bir karakteri, bir karakter dizisi ni ve sayısal değerleri dizeye dönüştürmek için birkaç statik valueOf metodu sağlar. Bu methods have the same name valueOf with farklı argüman türleri char, char[], double, long, int ve float. Örneğin, bir double

değerini bir dizeye dönüştürmek için String.valueOf(5.44) kullanın. The

return değeri, '5', '.' karakterlerinden oluşan bir string'dir. ‘4’, and ‘4’.

StringBuilder ve StringBuffer

StringBuilder/StringBuffer sınıfı, String sınıfına alternatif bir seçenektir. Genel olarak, bir StringBuilder/StringBuffer, bir string'in kullanıldığı her yerde kullanılabilir. StringBuilder/StringBuffer daha String'den daha esnek. Ekleyebilir, yerleştirebilir veya ekleyebilirsiniz Yeni içerikler bir dize tamponuna eklenir, oysa bir String nesnesinin değeri dize oluşturulduktan sonra sabittir.

`StringBuilder` Yapıcılar

java.lang.StringBuilder	
+StringBuilder()	16 kapasiteye sahip boş bir dize oluşturucu oluşturur.
+StringBuilder(kapasite: int)	Belirtilen kapasiteye sahip bir dize oluşturucu oluşturur.
+StringBuilder(s: String)	Belirtilen dizeye sahip bir dize oluşturucu oluşturur.

Builder'daki Dizeyi Değiştirme

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Bu string builder'a bir char dizisi ekler.
+append(data: char[], offset: int, len: int): StringBuilder	Bu string builder'a verilerdeki bir alt diziyi ekler.
+append(v: *aPrimitiveType*): StringBuilder	Bu builder'a bir ilkel tür değerini string olarak ekler.
+append(s: String): StringBuilder	Bu string builder'a bir string ekler.
+delete(startIndex: int, endIndex: int): StringBuilder	startIndex'ten endIndex'e kadar olan karakterleri siler.
+deleteCharAt(index: int): StringBuilder	Belirtilen indeksteki bir karakteri siler.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Belirtilen indekste, dizideki verilerin bir alt dizisini builder'a ekler.
+insert(offset: int, data: char[]): StringBuilder	Bu builder'a, konum ofsetinde veri ekler.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Bu builder'a string'e dönüştürülmüş bir değeri ekler.
+insert(offset: int, s: String): StringBuilder	Bu builder'a, konum ofsetinde bir string ekler.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Bu builder'daki karakterleri startIndex'ten endIndex'e kadar belirtilen string ile değiştirir.
+reverse(): StringBuilder	Builder'daki karakterleri tersine çevirir.
+setCharAt(index: int, ch: char): void	Bu builder'da belirtilen indekste yeni bir karakter ayarlar.

Örnekler

stringBuilder.append("Java");

stringBuilder.insert(11, "HTML ve ");

stringBuilder.delete(8, 11) yapısı "Welcome" olarak değiştirir. Java.

stringBuilder.deleteCharAt(8) yapısı "Welcome o Java." olarak değiştirir. Welcome o Java.

stringBuilder.reverse() yapısı "avaJ ot" olarak değiştirir. emocleW.

stringBuilder.replace(11, 15, "HTML")

yapısı "Welcome to HTML." olarak değiştirir.

stringBuilder.setCharAt(0, 'w') yapıcının "welcome" olarak ayarlanmasını sağlar Java'ya.

toString, capacity, length, setLength ve charAt Yöntemleri

java.lang.StringBuilder	
+toString(): String	String builder'dan bir string nesnesi döner.
+capacity(): int	Bu dize oluşturucusunun kapasitesini döndürür.
+charAt(index: int): char	Belirtilen indeksteki karakteri döndürür.
+length(): int	Bu yapıcıdaki karakter sayısını döndürür.
+setLength(newLength: int): void	Bu yapıcıda yeni bir uzunluk ayarlar.
+substring(startIndex: int): String	startIndex'ten başlayarak bir alt dize döndürür.
+substring(startIndex: int, endIndex: int): String	startIndex'ten endIndex-1'e kadar bir alt dize döndürür.
+trimToSize(): void	String builder için kullanılan depolama boyutunu azaltır.

Problem: Palindromları Kontrol Etme Alfabetik Olmayan Karakterleri Yoksayma

Bu örnek, bir dizideki her harfin kaç kez tekrarlandığını sayan bir program verir.

Harfların büyük/küçük harf duyarlı olmadığını varsayın.

PalindromeIgnoreNonAlphanumeric

Çalıştır

Ek H

Regüler İfadeler

A *düzenli ifade* (kısaca *regex*) bir dize bir küme ile eşleşen bir deseni tanımlar. dizeler. Düzenli ifadeler,

dize manipülasyonları için güçlü bir araçtır. Düzenli ifadeleri kullanabilirsiniz

eşleştirme, değiştirme ve ayırma ifadeleri dizeler.

Ek H

Uygun Dize

```
"Java".matches("Java");
"Java".equals("Java");

"Java eğlencelidir".matches("Java.*")
"Java havalıdır".matches("Java.*")
"Java güçlü".matches("Java.*")
```

Ek H

Regular İfade

Sözdizimi

Regular Expression	*Matches*	*Example*
x	a specified character x	Java matches Java
.	any single character	Java matches J..a
(ab|cd)	ab or cd	ten matches t(en|im)
[abc]	a, b, or c	Java matches Ja[uvwx]a
[^abc]	any character except a, b, or c	Java matches Ja[^ars]a
[a-z]	a through z	Java matches [A-M]av[a-d]
[^a-z]	any character except a through z	Java matches Jav[^b-d]
[a-e[m-p]]	a through e or m through p	Java matches [A-G[I-M]]av[a-d]
[a-e&&[c-p]]	intersection of a-e with c-p	Java matches [A-P&&[I-M]]av[a-d]
\d	a digit, same as [0-9]	Java2 matches "Java[\\d]"
\D	a non-digit	$Java matches "[\\D][\\D]ava"
\w	a word character	Java1 matches "[\\w]ava[\\w]"
\W	a non-word character	$Java matches "[\\W][\\w]ava"
\s	a whitespace character	"Java 2" matches "Java\\s2"
\S	a non-whitespace char	Java matches "[\\S]ava"
*p**	zero or more occurrences of pattern *p*	aaaabb matches "a*bb" ababab matches "(ab)*"
p+	one or more occurrences of pattern *p*	a matches "a+b*" able matches "(ab)+.*"
p?	zero or one occurrence of pattern *p*	Java matches "J?Java" Java matches "J?ava"
p{n}	exactly n occurrences of pattern *p*	Java matches "Ja{1}.*" Java does not match ".{2}"
p{n,}	at least n occurrences of pattern *p*	aaaa matches "a{1,}" a does not match "a{2,}"
p{n,m}	between n and m occurrences (inclusive)	aaaa matches "a{1,9}" abb does not match "a{2,9}bb"

Dizeleri Değiştirme ve Bölme

java.lang.String
+matches(regex: String): boolean
+replaceAll(regex: String, replacement: String): String
+replaceFirst(regex: String, replacement: String): String
+split(regex: String): String[]

Bu dize, deseni eşleşiyorsa true döner.

Tüm eşleşen alt dizeleri değiştir en yeni bir dize döner.

İlk eşleşen alt dizeyi değiştiren yeni bir dize döndürür.

Eşleşmelerle bölünmüş alt dizelerden ol uşan bir dizi dize döndürür.

Ek H

Örnekler

```
String s = "Java Java Java".replaceAll("v\\w", "wi") ;

String s = "Java Java Java".replaceFirst("v\\w", "wi") ;

String[] s = "Java1HTML2Perl".split("\\d");
```